GEBZE TECHNICAL UNIVERSITY

Department of Computer Engineering

CSE 344 - System Programming

Homework #4 Report

Multi-Process and Multi-Threaded Concurrent Log Analysis Pipeline

Muhammed Korkmaz

Student ID: 220104004043

May 1, 2026

Contents

1 Introduction

The objective of this homework is to implement a concurrent log analysis pipeline in C for POSIX/Linux. The system reads multiple log files, parses valid log entries, routes them according to severity level, computes weighted keyword scores, separately tracks high-priority sources, and produces both a human-readable report and a binary checkpoint file.

The implementation combines two concurrency models:

- a multi-process architecture using `fork`, anonymous shared memory, pipes, and process-shared synchronization primitives;
- POSIX threads inside selected processes, including mutexes, condition variables, barriers, and thread-local storage.

All child processes are forked only by the parent process. No child process creates another process.

2 Implemented File Structure

The submitted source layout follows the required project structure:

```
Makefile
main.c
reader.c / reader.h
dispatcher.c / dispatcher.h
analyzer.c / analyzer.h
aggregator.c / aggregator.h
shm.c / shm.h
watchdog.c / watchdog.h
220104004043_report.pdf
```

The `Makefile` builds the project with:

```
gcc -Wall -Wextra -pthread -o analyzer main.c reader.c dispatcher.c \
    analyzer.c aggregator.c shm.c watchdog.c
```

3 System Architecture

The parent process acts as the coordinator. It parses command-line arguments, loads the configuration files, initializes shared memory regions and all process-shared synchronization primitives, creates heartbeat pipes, forks the worker processes, starts the watchdog thread, waits for all children, and prints the final summary.

The process topology is:

- **Reader Processes:** one process per log file. Each reader creates multiple reader threads and one parser thread.

- **Dispatcher Process:** consumes Region A and routes entries to the correct Region B buffer and optionally Region D.
- **Analyzer Processes:** four processes, one for each severity level: ERROR, WARN, INFO, and DEBUG. Each analyzer creates a worker thread pool.
- **Aggregator Process:** waits for completed level results, computes high-priority score, writes the text report and binary checkpoint.
- **Watchdog Thread:** runs inside the parent process and monitors reader heartbeat pipes using `select()`.

4 Shared Memory Design

All shared memory regions are allocated by the parent process before any fork using:

```
mmap(NULL, size, PROT_READ | PROT_WRITE,
     MAP_SHARED | MAP_ANONYMOUS, -1, 0);
```

All process-shared mutexes and condition variables are also initialized by the parent. Children inherit the initialized state and never re-initialize these primitives.

4.1 Region A: Dispatcher Input Queue

Region A is a circular queue of `log_entry_t`. It is written by Reader parser threads and consumed by the Dispatcher process. It contains:

- `pthread_mutex_t input_mutex`
- `pthread_cond_t not_full_a`
- `pthread_cond_t not_empty_a`
- `eof_count_per_level[4]`
- circular-buffer metadata and data area

4.2 Region B: Per-Level Analyzer Buffers

There are four independent Region B buffers, one per severity level. The Dispatcher is the only writer. The matching Analyzer process is the only reader.

4.3 Region C: Results Area

Region C stores the four `level_result_t` structures, process-shared result mutex and condition variable, and supplementary unnamed process-shared semaphores.

The Aggregator uses `pthread_cond_timedwait()` on the process-shared condition variable as the canonical waiting mechanism. The semaphores are kept as supplementary signals, which is accepted by the clarification message.

4.4 Region D: High-Priority Buffer

Region D is a bounded circular queue for entries whose `SOURCE` is listed in the priority filter file. The Dispatcher writes these entries and the Aggregator drains them while also waiting for final level results.

5 Log Entry and EOF Sentinel Representation

The log entry type includes an explicit EOF marker flag:

```
typedef struct
{
    char timestamp[32];
    char level[8];
    char source[MAX_SOURCE];
    char message[MAX_MSG];
    int level_index; /* 0=ERROR,1=WARN,2=INFO,3=DEBUG */
    int is_eof;      /* 1 for EOF sentinel entries, 0 for normal log entries */
} log_entry_t;
```

Normal log entries have `is_eof = 0`. EOF sentinels have `is_eof = 1`, and `level_index` stores the corresponding severity level. This avoids overloading negative severity values and matches the official clarification.

6 Reader Process

Each Reader process creates `T` reader threads and one parser thread. Reader threads divide the input file into byte ranges. If a range starts in the middle of a line, the thread skips the partial line so that malformed duplicate fragments are not produced.

Reader threads parse lines defensively and push valid entries into an internal bounded buffer protected by a default pthread mutex and condition variables. This internal buffer intentionally does not use semaphores.

The parser thread consumes the internal buffer and pushes entries into Region A under the process-shared Region A mutex. After all reader threads finish and the internal buffer is drained, it sends four EOF sentinels, one for each severity level.

7 Dispatcher Process

The Dispatcher consumes entries from Region A using `pthread_cond_timedwait()` with the configured timeout. For each normal entry:

- it checks whether the source is high-priority;
- it forwards high-priority entries to Region D;

- it routes every entry to the matching Region B severity buffer.

EOF forwarding is based on EOF sentinels actually consumed by the Dispatcher. This preserves queue ordering and prevents a level EOF from being forwarded before normal entries that are still waiting in Region A.

8 Analyzer Processes

There are four Analyzer processes. Each Analyzer owns one severity level and creates `W` worker threads.

Each worker:

- allocates a heap-based per-keyword score array;
- stores it with `pthread_setspecific()`;
- consumes entries from the matching Region B buffer;
- counts overlapping keyword occurrences using a sliding-window search;
- multiplies counts by the severity weight;
- tracks per-thread weighted contribution and per-source hit counts.

The severity weights are:

Level	Weight
ERROR	4
WARN	3
INFO	2
DEBUG	1

8.1 TLS and Reporting Thread

The implementation uses `pthread_key_t` for worker-local keyword scores. The TLS destructor flushes each worker's per-keyword scores into Region C under `result_mutex`. To avoid incomplete summaries, the lowest Linux TID reporting worker waits until all other workers have flushed their TLS values before setting the final `ready` flag.

The reporting thread is selected using:

```
pid_t tid = (pid_t)syscall(SYS_gettid);
```

`pthread_self()` is not used for this decision.

9 Aggregator Process

The Aggregator waits for the four level results with `pthread_cond_timedwait()` on Region C. While waiting, it also drains the high-priority Region D buffer so the Dispatcher cannot deadlock on a full priority queue.

For `HIGH_PRIORITY_SCORE`, the Aggregator applies the same scoring rules as Analyzer workers: case-sensitive overlapping keyword counts multiplied by the severity weight of the entry.

The human-readable output is sorted by total weighted score descending. The binary checkpoint is written to a temporary file first and then atomically renamed:

```
rename(tmp_path, cfg->binary_path);
```

10 Watchdog Thread

The parent process starts one watchdog thread after all child processes are forked. It monitors Reader heartbeat pipes using `select()` with a three-second timeout. It prints progress only to `stderr`, for example:

```
[WATCHDOG] Progress at T+3s: Reader0=2 Reader1=0 children_alive=0
```

11 SIGINT Handling and Cleanup

The SIGINT handler is async-signal-safe. It only sets a `volatile sig_atomic_t` flag and writes a short message with `write(2)`. The parent main loop detects this flag, terminates child processes, reaps them, stops the watchdog thread, and unmaps shared memory.

Normal shutdown destroys process-shared mutexes, condition variables, and semaphores in the parent after all child processes exit.

12 Testing

Because this text report cannot embed live terminal screenshots, the relevant terminal commands and outputs are included as textual evidence. The tests were run on the project directory before creating the submission archive.

12.1 Compilation Test

```
$ make clean
rm -f analyzer *.o *.tmp

$ make
gcc -Wall -Wextra -pthread -o analyzer main.c reader.c dispatcher.c analyzer.c
    aggregator.c shm.c watchdog.c
```

No compiler warnings were produced.

12.2 Functional Pipeline Test

The following temporary test data was used:

```
[2026-01-01 00:00:00] [ERROR] [src1] aaaa
[2026-01-01 00:00:01] [WARN] [src2] errorerror
[2026-01-01 00:00:02] [DEBUG] [src1] aa
[2026-01-01 00:00:03] [INFO] [bad_src] aaaa
[2026-01-01 00:00:04] [BOGUS] [src3] aaaa
not a log line
[2026-01-01 00:00:05] [INFO] [src4] aaerror
[2026-01-01 00:00:06] [ERROR] [src1] fail kill
```

The invalid source `bad_src`, invalid level `BOGUS`, and malformed line were skipped. The run command was:

```
$ ./analyzer -c cfg.txt -f filter.txt -k aa,error,fail,kill \
    -t 2 -w 3 -a 8 -b 8 -d 4 -T 2 -o out.txt -O out.bin
```

Observed final summary:

```
SYSTEM SUMMARY
Keywords  : aa,error,fail,kill
Log files : 2
Total entries  : 5
Total weighted : 31.0
High-priority  : 21.0
  ERROR : 2 entries, score: 20.0
  WARN  : 1 entries, score: 6.0
  INFO  : 1 entries, score: 4.0
  DEBUG : 1 entries, score: 1.0
Program terminated successfully.
```

12.3 Human-Readable Output File

```
KEYWORD_LIST: aa,error,fail,kill
FILES: 2
TOTAL_WEIGHTED_SCORE: 31.0
HIGH_PRIORITY_SCORE: 21.0
# Levels sorted by total_weighted_score DESC
LEVEL  ENTRIES  WEIGHTED_SCORE  aa  error  fail  kill
ERROR         2           20.0    12.0     0.0     4.0     4.0
WARN          1            6.0     0.0     6.0     0.0     0.0
INFO          1            4.0     2.0     2.0     0.0     0.0
DEBUG         1            1.0     1.0     0.0     0.0     0.0
# Top-3 sources per level
ERROR   src1:2
```

```
WARN    src2:1
INFO    src4:1
DEBUG   src1:1
# Per-thread contributions (weighted score)
ERROR   thread_0:20.0
WARN    thread_0:6.0
INFO    thread_0:4.0
DEBUG   thread_0:1.0
```

12.4 Binary Checkpoint Verification

The binary file header begins with the required magic value. On a little-endian Linux machine, `0xC5E3440B` appears as `0b44e3c5`:

```
$ xxd -g 4 -l 16 out.bin
00000000: 0b44e3c5 01000000 04000000 04000000  .D..............
```

12.5 Valgrind Smoke Test

A short Valgrind run with child tracing was also executed:

```
$ valgrind --trace-children=yes --leak-check=full --error-exitcode=77 \
    ./analyzer -c cfg.txt -f filter.txt -k error,fail,aa \
    -t 1 -w 1 -a 8 -b 8 -d 4 -T 2 -o out.txt -O out.bin
```

Relevant result:

```
ERROR SUMMARY: 0 errors
definitely lost: 0 bytes
All heap blocks were freed -- no leaks are possible
```

13 Challenges and Solutions

13.1 EOF Ordering

An EOF counter alone can become visible before the Dispatcher consumes all earlier normal entries. To avoid premature analyzer termination, EOF forwarding is based on sentinels consumed from Region A, preserving queue order.

13.2 Priority Buffer Deadlock

Region D is bounded. If the Aggregator waited only for all level results before reading Region D, the Dispatcher could block on a full priority buffer. The Aggregator therefore drains Region D while also waiting for Region C readiness.

13.3 TLS Destructor Ordering

If the reporting worker reads Region C before other TLS destructors flush their scores, the summary can be incomplete. The implementation uses an explicit active-worker counter and condition variable so the lowest-TID reporting worker waits until the other TLS destructors have completed.

14 Conclusion

The implemented system satisfies the required multi-process and multi-threaded pipeline design. It uses anonymous shared memory, process-shared mutexes and condition variables, unnamed process-shared semaphores, Reader-side pthread synchronization, Analyzer TLS, barriers, timed condition waits, watchdog monitoring, atomic binary output, and SIGINT cleanup. The final tests showed correct scoring, malformed-line handling, high-priority aggregation, binary checkpoint format, and clean compilation with `-Wall -Wextra -pthread`.